

IMPORTS & FUNCTIONS

INFO1 Mission 3 (S4)

Jean-Luc Bibaud



Imports

What does importing actually do?

How do you partially import from a module in Python?

```
from math import pi, tan
```

```
print(pi)
```

```
→ 3.141592653589793
```

```
print(cos(90 * pi))
```

```
→ NameError: name 'cos' is not defined
```

```
import math
```

```
print(pi)
```

```
→ NameError: name 'pi' is not defined
```

```
print(math.pi)
```

```
→ 3.141592653589793
```

```
print(cos(90 * math.pi))
```

```
→ NameError: name 'cos' is not defined
```

FUNCTIONS

Functions reminder

```
def NAME (PARAMETERS) :  
    STATEMENTS  
    return VALUE      # optional
```

Without a return, the function gives back None.

Invoking / calling

= executing the code of that function

```
NAME (ARGUMENTS)
```

You already do this! `print()`, `range()`, `type()`, `len()` ... are all functions.

Function variables are local (encapsulation)

```
lebleu = "une couleur"
print(lebleu)          # une couleur

def lebapteme():
    lebleu = "con"
    print(lebleu)

lebapteme()            # con
print(lebleu)          # une couleur !
```

The lebleu inside the function is a **different variable**. It disappears when the function ends — the outside one is untouched.

⚠ Loops are NOT encapsulated

```
for i in range(5):
    print(i)
print(i)              # 4
```

i still exists after the loop. Only functions get their own local variables.

Arguments and return



Do it many times, with different values

```
def area(w, h):  
    return w * h  
  
area(2, 3)    # 6  
area(5, 4)    # 20
```

A return can be the input of a function

```
x = area(1, 2)    # 2  
area(x, 3)        # 6  
  
# or in one go:  
area(area(1, 2), 3) # 6
```

WATCH OUT

Common beginner mistakes

print instead of return

```
def add(a, b):  
    print(a + b)  
x = add(1, 2)  
# x is None
```

If you need the value later, return it.

Forgetting the ()

```
lebapteme      # no call  
lebapteme()    # runs it
```

Without parentheses you only name the function.

Local variable used outside

```
def f():  
    y = 5  
f()  
print(y)      # NameError
```

Return the value instead.

Code after return

```
def f():  
    return 1  
    print("hi")  
# print never runs
```

return ends the function immediately.

Calling before defining

```
hello()      # NameError  
def hello():  
    print("hi")
```

Python reads top to bottom: define first.

Naming a file random.py

```
import random  
random.randint(1, 6)  
# AttributeError
```

Your file hides the real module. Rename it.